

```
#Program 1: Generate Raw Mock Data

import pandas as pd
import numpy as np

raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28"
    ]
}

df = pd.DataFrame(raw_data)

print("Raw Data")
print(df)
```

Raw Data				
	Employee_Name	Department	Salary	Join_Date
0	Alice	HR	50000.0	2022-01-15
1	Bob	Engineering	85000.0	2021-06-20
2	Charlie	HR	NaN	2023-03-11
3	Diana	Engineering	92000.0	2020-11-01
4	Evan	Marketing	60000.0	2024-02-28

```
#Program 2: Data Preprocessing

import pandas as pd
import numpy as np

raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28"
    ]
}

df = pd.DataFrame(raw_data)

df["Employee_Name"] = df["Employee_Name"].str.strip()
df["Salary"] = df["Salary"].fillna(df["Salary"].median())
df["Join_Date"] = pd.to_datetime(df["Join_Date"])
df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year

print(df)
```

	Employee_Name	Salary	Join_Date	Years_of_Service
0	Alice	50000.0	2022-01-15	4
1	Bob	85000.0	2021-06-20	5
2	Charlie	72500.0	2023-03-11	3
3	Diana	92000.0	2020-11-01	6
4	Evan	60000.0	2024-02-28	2

```
#Program 3: Data Aggregation

import pandas as pd

data = {
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Employee_Name": ["Alice", "Bob", "Charlie", "Diana", "Evan"],
    "Salary": [50000, 85000, 67500, 92000, 60000]
}

df = pd.DataFrame(data)

dept_summary = df.groupby("Department").agg(
    Avg_Salary=("Salary", "mean"),
    Total_Employees=("Employee_Name", "count")
).reset_index()

print(dept_summary)
```

	Department	Avg_Salary	Total_Employees
0	Engineering	88500.0	2
1	HR	58750.0	2
2	Marketing	60000.0	1

#Program 4: Data Visualization

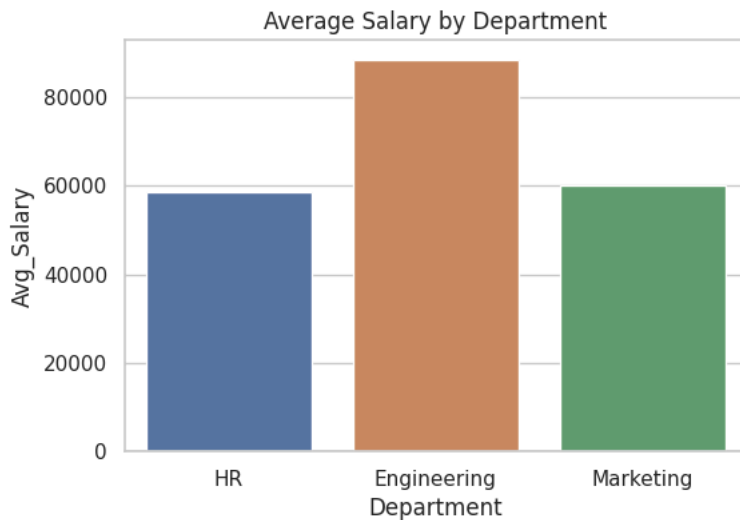
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    "Department": ["HR", "Engineering", "Marketing"],
    "Avg_Salary": [58750, 88500, 60000]
}

df = pd.DataFrame(data)

sns.set_theme(style="whitegrid")

plt.figure(figsize=(6,4))
sns.barplot(data=df, x="Department", y="Avg_Salary", hue="Department", legend=False)
plt.title("Average Salary by Department")
plt.show()
```



#Program 5: Creating a Sample DataFrame

```
import pandas as pd

# Create a Sample DataFrame
df = pd.DataFrame()

df["Name"] = ["John", "Emma", "Liam", "Olivia"]
df["Age"] = [20, 19, 21, 18]
df["Student"] = [True, True, False, True]

print(df)
```

	Name	Age	Student
0	John	20	True
1	Emma	19	True
2	Liam	21	False
3	Olivia	18	True

#Program 6: Adding a Row to a DataFrame

```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    "Name": ["John", "Emma", "Liam", "Olivia"],
    "Age": [20, 19, 21, 18],
    "Student": [True, True, False, True]
})

# Add a New Row
new_row = pd.DataFrame(
    [{"Sophia", 22, False}],
    columns=["Name", "Age", "Student"]
)

df = pd.concat([df, new_row], ignore_index=True)

print(df)
```

	Name	Age	Student
0	John	20	True
1	Emma	19	True

```
2   Liam    21    False
3   Olivia  18     True
4   Sophia 22    False
```

```
#Data Analysis with SciPy
```

```
#Program 7: Importing Required Libraries
```

```
import numpy as np
import pandas as pd
```

```
print("Libraries Imported Successfully")
```

```
Libraries Imported Successfully
```

```
#Program 8: Measures of Central Tendency using SciPy
```

```
import numpy as np
from scipy import stats
```

```
data = [10, 20, 30, 40, 50]
```

```
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Mode:", stats.mode(data))
```

```
Mean: 30.0
Median: 30.0
Mode: ModeResult(mode=np.int64(10), count=np.int64(1))
```

```
#Program 9: Probability Distribution Analysis using SciPy
```

```
from scipy.stats import norm
```

```
probability = norm.cdf(85, loc=70, scale=10)
```

```
print("Probability:", probability)
```

```
Probability: 0.9331927987311419
```

```
#Program 10: Hypothesis Testing using SciPy
```

```
from scipy import stats
```

```
data = [22, 25, 19, 24, 28, 30]
```

```
t_stat, p_value = stats.ttest_1samp(data, 25)
```

```
print("T-Statistic:", t_stat)
print("P-Value:", p_value)
```

```
T-Statistic: -0.2049800154226962
P-Value: 0.8456711339658732
```

```
#Program 11: Correlation Analysis using SciPy
```

```
from scipy.stats import pearsonr
```

```
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
```

```
corr, p_value = pearsonr(x, y)
```

```
print("Correlation:", corr)
```

```
Correlation: 1.0
```

```
#Program 12: Linear Algebra Operations using SciPy
```

```
from scipy import linalg
```

```
A = [[3, 2],
      [1, 2]]
```

```
B = [5, 5]
```

```
solution = linalg.solve(A, B)
```

```
print("Solution:", solution)
```

```
Solution: [0.  2.5]
```

```
#Program 13: Optimization using SciPy
```

```
from scipy.optimize import minimize
```

```
def objective(x):  
    return x**2 + 4  
  
result = minimize(objective, x0=5)  
  
print("Optimal Value:", result.x)
```

Optimal Value: [-2.62955131e-08]

#Scikit Library in Python

#Program 14: Logistic Regression using Scikit-learn

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load Iris Dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict and Evaluate
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

Accuracy: 1.0

#Program 15: K-Means Clustering using Scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load Iris Dataset
iris = load_iris()

# Create K-Means Model
kmeans = KMeans(n_clusters=3)

# Train Model
kmeans.fit(iris.data)

# Display Cluster Labels
print("Cluster Labels:")
print(kmeans.labels_)
```

```
Cluster Labels:  
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 0 0 0 0 2 0 0 0 0  
0 2 2 0 0 0 0 2 0 2 0 2 0 0 2 2 0 0 0 0 0 2 0 0 0 0 2 0 0 0 2 0 0
```